

分治之美

排序与二分 · 7题 · C++ & Python 双语对照

从本章开始，小鲁正式进入**算法**的世界。前面的语法和数据结构是工具，算法才是让程序"聪明"起来的灵魂。"排序——把混乱变成有序——是计算机科学中最基础也最深刻的主题。快排、归并、二分.....这些经典算法经过几十年的打磨，至今仍是每个程序员的基本功。"小华严肃地说。

本章7道题分三个板块：**排序模板**（快排+归并）——理解分治思想并将模板变成肌肉记忆；**排序应用**（快速选择+逆序对）——在排序过程中顺便解决更复杂的问题；**二分查找**（整数+实数）——利用有序性将搜索从 $O(n)$ 降到 $O(\log n)$ ，这是算法优化的第一思维。

本章角色

-  **小鲁**（进阶者）· 从语法期毕业，进入算法训练营——分治、排序、二分
-  **小华**（算法队长）· "快排模板写100遍——背下来、默出来、成为肌肉记忆"
-  **小栋**（工程师）· "为什么不能直接调用sort()?"——理解算法原理和工程实践的关系

⚡ 排序与二分核心模板

- **快排模板**：取中值基准→do-while双指针→交换→递归 $[l, j]$ 和 $[j+1, r]$
- **归并模板**：二分递归→双指针合并→复制回原数组。稳定 $O(n \log n)$
- **整数二分**：左边界 $mid = l + r >> 1$, $r = mid$ 。右边界 $mid = l + r + 1 >> 1$, $l = mid$
- **实数二分**：while($r - l > \text{eps}$) $mid = (l + r) / 2$ 。比整数二分简单——不需处理+1/-1
- **Python**：sort()/sorted()快速排序，bisect_left/right二分。标准库是工程首选

NQ106 快速排序 AcWing 785

"排序——这是计算机科学里研究得最透彻的问题之一。"小华郑重地说，"给你 n 个乱序的整数，用快速排序把它们排好。快排的核心思想是'分治'——选一个基准数，把比它小的放左边，大的放右边，然后左右分别递归排序。这个模板你会写100遍——背下来，默出来，成为肌肉记忆。"小鲁试了试："do-while循环和指针的配合确实巧妙，不小心就会无限循环....."

- 输入** 第一行整数 n ，第二行 n 个整数。
- 输出** 升序排列的 n 个整数，空格分隔。
- 范围** $1 \leq n \leq 100000$ ，所有整数在 $1 \sim 10^9$ 范围内

输入

5
3 1 2 4 5

输出

1 2 3 4 5

思路 快排模板：选基准 $x = arr[l+r \gg 1]$ ， $i = l - 1, j = r + 1$ 双指针向中间移动， i 找 $\geq x$ 的， j 找 $\leq x$ 的，交换，直到 $i \geq j$ ，然后递归 $[l, j]$ 和 $[j+1, r]$ 。时间 $O(n \log n)$ 平均，最坏 $O(n^2)$ 。注意边界：用do-while避免死循环，用 j 而非 $i-1$ 防止无限递归。

技巧 快排背板要点：1)取中间值作基准避免最坏情况；2) i 和 j 初始为 $l-1$ 和 $r+1$ 配合do-while；3)递归用 j 和 $j+1$ （不用 i ）。Python版`sorted()`内部用Timsort，实践中永远比手写快排好。但手写快排是理解分治思想的必修课。

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

long long *numbers;

void quicksort(long long nums[], int left, int right)
{
    if (left >= right)
        return;
    //选取分割数
    long long target = nums[left]; //选取第一个快排的分割数
    int i = left - 1, j = right + 1;
    while (i < j)
    {
        do
            i++;
        while (nums[i] < target); //i指针定位
        do
            j--;
        while (nums[j] > target); //j指针定位
        if (i < j)
            swap(nums[i], nums[j]); //交换
    }
    //分治
    quicksort(nums, left, j); //j左边的都是不比Target大的
    quicksort(nums, j + 1, right); // j右边的都比Target大
}

int main()
{
    //!cin\cout提速
    ios::sync_with_stdio(false); //来打消iostream的输入输出缓存
    cin.tie(0); //cin.tie(0)来解除cin与cout的绑定,0表示NULL

    int n;
    //创建动态数组
    cin >> n;
    numbers = new long long[n];
    for (int i = 0; i < n; i++)
        cin >> numbers[i];
    quicksort(numbers, 0, n - 1); //注意边界

    for (int i = 0; i < n; i++)
        cout << numbers[i] << " ";

    delete numbers;
    return 0;
}

```

Python

Python solution

NQ107 第k个数 AcWing 786

"如果是找第k小的数——而不是排序整个数组呢？"小栋问。小华说："用快排的变种——快速选择算法。每次划分后，看第k个数落在左半边还是右半边，只递归那一边。平均 $O(n)$ 时间找到第k小，比先排序 $O(n \log n)$ 再取第k个快。这就是'不需要的信息就不处理'的思维——算法优化的本质就是只做必要的工作。"

输入

第一行n和k，第二行n个整数。

输出

第k小的数。

范围

$1 \leq n \leq 100000, 1 \leq k \leq n$

输入

5 3
2 4 1 5 3

输出

3

思路 快速选择：在快排划分基础上，设左半长度为 $sl=j-l+1$ 。若 $k \leq sl$ 则在左半递归找第k小；否则在右半递归找第 $k-sl$ 小。平均 $O(n)$ ，最坏 $O(n^2)$ （可通过随机选基准优化）。Python直接用`nth_element`或排序后取`arr[k-1]`。

技巧 快速选择的核心是partition后的决策——只递归一边。这和二分查找的思想一样：每一步排除一半候选。最坏 $O(n^2)$ 可以用随机化基准数来规避——C++的`nth_element`就是这个算法。

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

long long *numbers;

long long quickFind(long long nums[], int left, int right, int k)
{
    if (left == right)
        return nums[left]; //只有一个元素，左开右闭
    //选取分割数
    long long target = nums[left]; //选取第一个快排的分割数
    int i = left - 1, j = right + 1;
    while (i < j)
    {
        do
            i++;
        while (nums[i] < target); //i指针定位
        do
            j--;
        while (nums[j] > target); //j指针定位
        if (i < j)
            swap(nums[i], nums[j]); //交换
    }
    //判断i与k的关系
    int sumofLeft = j - left + 1; //计算左边区间数字个数，用于与k比较

    if (k <= sumofLeft) //分治
        quickFind(nums, left, j, k); //j左边的都是不比Target大的
    else
        quickFind(nums, j + 1, right, k - sumofLeft); //j右边的都比Target大
}

int main()
{
    //!cin\cout提速
    ios::sync_with_stdio(false); //来打消iostream的输入输出缓存
    cin.tie(0); //cin.tie(0)来解除cin与cout的绑定,0表示NULL

    int n, k;
    //创建动态数组
    cin >> n >> k;
    numbers = new long long[n];
    for (int i = 0; i < n; i++)
        cin >> numbers[i];
    //查找第k小数，输出之
    cout << quickFind(numbers, 0, n - 1, k) << endl; //注意边界

    delete numbers;
    return 0;
}

```

Python

Python solution

"除了快排，还有一种经典排序——归并排序。"小华画出一个分叉图，"先把数组不断对半分，分到只有一个元素（自然有序），然后从下往上两两合并有序子数组。归并排序的优点是稳定且时间复杂度始终 $O(n \log n)$ ，缺点是需要额外 $O(n)$ 空间。快排vs归并——这是算法课永恒的话题。"

输入 第一行n，第二行n个整数。

输出 升序排列的n个整数。

范围 $1 \leq n \leq 100000$

输入

5
3 1 2 4 5

输出

1 2 3 4 5

思路 归并三步骤：1)递归划分 $mid=l+r>>1$ ，`merge_sort(l,mid)`和`merge_sort(mid+1,r)`；2)双指针合并两个有序子数组到tmp；3)复制tmp回原数组。稳定排序，时间 $O(n \log n)$ ，空间 $O(n)$ 。逆序对问题可以天然嵌入归并过程。

技巧 归并是稳定排序——相等元素的相对位置不变。快排不稳定。Python用`sorted()`（稳定），C++用`stable_sort`（归并）和`sort`（快排）。理解归并对后续学习逆序对、CDQ分治至关重要。

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
const int N = 100007;
int n;
int numbers[N], tmp[N];

void mergeSort(int nums[], int left, int right)
{
    if (left >= right) return;

    int mid = left + (right - left) / 2; // 防止溢出

    mergeSort(nums, left, mid), mergeSort(nums, mid + 1, right); // 先调用归并排序
    // 合二为一
    int k = 0, i = left, j = mid + 1; // 左右两部分数组的起点

    while (i <= mid && j <= right) // 扫描还未抵达终点
        if (nums[i] <= nums[j]) // 把小的放到tmp数组里面
            tmp[k++] = nums[i++];
        else tmp[k++] = nums[j++];

    while (i <= mid) tmp[k++] = nums[i++]; // 把左边剩余的部分插入tmp
    while (j <= right) tmp[k++] = nums[j++]; // 把右边剩余的部分插入tmp

    // 把tmp的部分复制回数组
    for (i = left, k = 0; i <= right; i++, k++) nums[i] = tmp[k];
}

int main()
{
    // 数据大, 使用printf和scanf
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &numbers[i]);

    mergeSort(numbers, 0, n - 1);

    for (int i = 0; i < n; i++) printf("%d ", numbers[i]);

    return 0;
}

```

Python

Python solution

NQ109 逆序对的数量 AcWing 788

"统计数组中逆序对的数量——即 $a[i]$ 的对数。"小华说, "暴力两重循环 $O(n^2)$ 太慢。巧妙的方法? 在归并排序的合并过程中统计! 当左半的 $a[i]$ 大于右半的 $a[j]$ 时, 左半从 i 到 mid 的所有数都大于 $a[j]$, 逆序对增加 $mid - i + 1$ 个。排序+统计一气呵成。注意答案可能超过 int 范围——用 $long\ long$!"

输入

第一行 n , 第二行 n 个整数。

输出

逆序对总数。

范围

 $1 \leq n \leq 100000$, 答案可能超 int

输入

6
2 3 4 5 6 1

输出

5

思路 归并排序嵌入计数：当 $a[i] > a[j]$ 时， $\text{cnt} += \text{mid} - i + 1$ （左半剩余元素全部形成逆序对）。归并过程中自然完成了逆序对统计。时间复杂度 $O(n \log n)$ ，比暴力 $O(n^2)$ 快4个数量级（ $n=10^5$ 时）。逆序对数量是衡量数组“混乱程度”的指标。

技巧 C++必须用long long存储cnt—— $n=10^5$ 时逆序对最多约 5×10^9 超过int。Python自带大整数。这道题是归并排序最重要的应用之一——理解‘在归并过程中顺便统计’是分治算法的精髓。

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
typedef long long LL;

const int N = 100007;
int n;
int numbers[N], tmp[N];
//最大的数可能会是  $n^2/2$ 
LL mergeSort(int nums[], int left, int right)
{
    if (left >= right) return 0; //如果左指针越过右指针, 则返回

    int mid = left + (right - left) / 2; //防止溢出

    LL result = mergeSort(nums, left, mid) + mergeSort(nums, mid + 1, right); //先调用递归排序
    //合二为一
    int k = 0, i = left, j = mid + 1; //左右两部分数组的起点

    while (i <= mid && j <= right) //扫描还未抵达终点
        if (nums[i] <= nums[j]) //把小的放到tmp数组里面
            tmp[k++] = nums[i++];
        else {
            //计算逆序对的个数 mid - i + 1
            result += mid - i + 1;
            tmp[k++] = nums[j++];
        }

    while (i <= mid) tmp[k++] = nums[i++]; //把左边剩余的部分插入tmp
    while (j <= right) tmp[k++] = nums[j++]; //把右边剩余的部分插入tmp

    //把tmp的部分复制回数组
    for (i = left, k = 0; i <= right; i++, k++) nums[i] = tmp[k];

    return result;
}

int main()
{
    //数据大, 使用printf和scanf
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &numbers[i]);

    cout << mergeSort(numbers, 0, n - 1) << endl;

    return 0;
}

```

Python

Python solution

NQ110 数的范围 AcWing 789

"在一个升序数组里, 某个数出现了多次——请找出它第一次和最后一次出现的位置。"小栋出题。小鲁说: "遍历一遍就行!" 小华摇头: "数组有 10^5 个元素, 查询 10^4 次—— $O(n \times q)$ 要 10^9 次操作, 超时。需要用二分

—— $O(\log n)$ 每次查询。二分的精髓：用mid+1或mid不停缩小范围，直到找到边界。整数二分的边界处理是90%的人都会写错的地方。”

输入 第一行n和q，第二行n个升序整数，然后q行每行一个查询k。

输出 每个查询输出k的起始和终止位置，不存在输出-1 -1。

范围 $1 \leq n \leq 100000$, $1 \leq q \leq 10000$

输入	输出
6 3	3 4
1 2 2 3 3 4	1 2
3	-1 -1
2	
5	

思路 二分查找左右边界：左边界→while(l>1; if(a[mid]>=k) r=mid else l=mid+1。右边界→while(l>1; if(a[mid]<=k) l=mid else r=mid-1。注意右边界二分的mid要+1避免死循环。Python可用bisect_left和bisect_right。

技巧 整数二分最关键的差异：左边界mid=l+r>>1，右边界mid=l+r+1>>1（不+1会死循环）。记住：l=mid时mid要+1，r=mid时不用。二分是OI/ACM中最基础也是最容易写错的算法——花时间理解透彻值得。

C++

```

#include <iostream>
#include <algorithm>
using namespace std;
const int N = 100007;

int nums[N];

int main()
{
    // 第一行包含整数n和q, 表示数组长度和询问个数。
    int n, q;
    scanf("%d%d", &n, &q);
    // 第二行包含n个整数 (均在1~10000范围内), 表示完整数组。
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);
    // 接下来q行, 每行包含一个整数k, 表示一个询问元素。
    while(q--){
        //读入要查找的数, 并且二分查找其范围
        int k;
        scanf("%d",&k);// 读入k

        int left = 0, right = n-1;//初始化left和right
        while (left < right) //模板2
        {
            int mid = left + right>>1;
            if (nums[mid]>=k) right = mid;//模板1,
            else left = mid +1;
        }
        //left就是左端点值
        //判断是否找得到
        if (nums[left]!=k) cout<<"-1 -1"<<endl;//找不到x
        else
        {
            cout<<left<<" ";
            //继续寻找右边界
            left=0, right=n-1;//重新初始化left,riight,进行第二
            次二分搜索
            while (left < right)
            {
                int mid = left+right+1>>1;
                if (nums[mid]<=k) left=mid;
                else right = mid -1;
            }
            cout<<left<<endl;
        }
    }

    return 0;

    // int bsearch_1(int left, int right)
    // {
    //     while (left < right)
    //     {
    //         int mid = left + right >> 1;
    //         if (check(mid)) right = mid;
    //         else left = mid + 1;
    //     }
    //     return left;
    // }

    // int bsearch_2(int left, int right)

```

Python

Python solution

```
// {
//     while (left < right)
//     {
//         int mid = left + right + 1 >> 1;
//         if (check(mid)) left = mid;
//         else right = mid - 1;
//     }
//     return left;
// }
```

NQ111 数的三次方根 AcWing 790

"整数二分搞定了，那实数二分呢？"小鲁问。"比如计算 n 的三次方根，精确到 10^{-6} 。"小华说，"实数二分比整数二分简单——不需要处理 $+1/-1$ 的边界。设定精度阈值（如 $1e-8$ ）， $\text{while}(r-l>1e-8)$ 不断对半分。浮点二分的优势：不需要纠结 mid 的取值，收敛极快—— $\log_2(10^8)\approx 27$ 步即可到目标精度。"

- 输入** 一个浮点数 n 。
- 输出** n 的三次方根，保留6位小数。
- 范围** $-10000 \leq n \leq 10000$

输入

1000.00

输出

10.000000

思路 实数二分：设 $l=-10000, r=10000$ （范围包含答案）， $\text{while}(r-l>1e-8)$: $\text{mid}=(l+r)/2$; $\text{if}(\text{mid}^3\geq n)$ $r=\text{mid}$ else $l=\text{mid}$ 。输出 r 保留6位小数（ $\text{printf}("%.6lf", r)$ ）。收敛步数 $=\log_2(\text{范围}/\text{精度})\approx\log_2(20000/1e-8)\approx 41$ 步——非常快。

技巧 实数二分不需要处理整数二分的 $\text{mid}+1$ 问题。直接 $\text{mid}=(l+r)/2$ 即可。精度通常比要求多两位（要求 $1e-6$ 则迭代到 $1e-8$ ）。C++用`double`，Python用`float`。注意 n 为负数时三次方根也为负。

C++

```
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    double number;
    scanf("%lf", &number);
    double left = -10000, right = 10000;
    while (right - left >= 1e-8)
    {
        double mid = (right + left) / 2; //二分
        if (mid * mid * mid >= number)
            right = mid;
        else
            left = mid;
    }
    //输出left
    printf("%lf", left);
}
```

Python

Python solution

NQ112 菱形 AcWing 727

"最后来一道'画图题'放松一下——打印一个菱形！"小华说，"看起来简单，但其实考察的是曼哈顿距离的几何直觉。菱形中心到四边的曼哈顿距离是 $n/2$ ，每个位置如果到中心距离 $\leq n/2$ 就填*否则填空格。这道题把几何和格式化输出结合起来——是算法竞赛中'观察规律→数学建模→代码实现'的完美微型案例。"

输入

一个奇数 n 。

输出

 n 行菱形图案，中心行有 n 个*。

范围

 $1 \leq n \leq 100$, n 为奇数

输入

5

输出

```
  *
 ***
*****
 ***
  *
```

思路 曼哈顿距离模型：设中心 $cx=cy=n/2$ 。对于每个位置 (i,j) ，如果 $abs(i-cx)+abs(j-cy) \leq n/2$ 则填*否则空格。这种'距离阈值'的判断方式在心形、沙漏等图案题中都通用。时间复杂度 $O(n^2)$ 。

技巧 曼哈顿距离 $=|x1-x2|+|y1-y2|$ 。菱形就是曼哈顿距离 $\leq$ 阈值的点集。换欧几里得距离就是圆形。理解这种几何距离与图形形状的关系，是计算机图形学的基础。

C++

```
#include <iostream>
#include <cmath>

using namespace std;

int main()
{
    int n;
    cin >> n;

    int cx = n / 2, cy = n / 2;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            if (abs(i - cx) + abs(j - cy) <= n / 2) cout <<
            '*';
        else cout << ' ';
        cout << endl;
    }

    return 0;
}
```

Python

Python solution

本章知识点总结

- 快排模板： $O(n \log n)$ 平均。do-while+双指针+分治递归。非稳定排序
- 归并模板： $O(n \log n)$ 保证。稳定排序。额外 $O(n)$ 空间。可在合并过程统计逆序对
- 快速选择：基于快排划分， $O(n)$ 平均找第k小。只递归一边——“不需要的信息就不处理”
- 整数二分：左边界和右边界模板不同——注意mid是否需要+1避免死循环
- 实数二分：无需处理+1/-1边界。迭代到目标精度+2位即可
- 厦大精神：自强不息。把这些模板练成肌肉记忆——它们是你算法之路的基石